

**Example 1 : A machine produces rice packets; probability of a defective packet = 0.02.
Find probability that in a batch of 100 packets, at most 5 are defective.**

```
from scipy.stats import binom
p = 0.02    # defect probability
n = 100     # sample size
prob_at_most_5 = binom.cdf(5, n, p)
print("Probability of at most 5 defective packets =", prob_at_most_5)
```

Example 2:

- **Disease prevalence = 1%**
 - **Test sensitivity = 0.95**
 - **Test specificity = 0.92**
- Compute the probability a patient has the disease given a positive test.**

```
preval = 0.01
sens = 0.95
spec = 0.92
P_pos = preval*sens + (1-preval)*(1-spec)
P_disease_given_pos = preval*sens / P_pos
print("P(Disease | Positive Test) =", round(P_disease_given_pos,4))
```

example 3:

**Example: Average lifetime of a light bulb = 1200 hours.
Find probability that a bulb lasts more than 1500 hours.**

```
from scipy.stats import expon

mean_life = 1200
lam = 1/mean_life
prob_more_1500 = expon.sf(1500, scale=1/lam)
print("Probability bulb lasts > 1500 hours =", prob_more_1500)
```

example 4:

Simulate sampling from a population and compute 95% CI of the mean using normal approximation.

```
import numpy as np
```

```

from scipy.stats import norm

# Population (unknown in real life)
population = np.random.normal(50, 10, 100000)

# Draw a sample
sample = np.random.choice(population, 200)
sample_mean = np.mean(sample)
sample_sd = np.std(sample, ddof=1)

# 95% confidence interval
z = norm.ppf(0.975)
lower = sample_mean - z * sample_sd / np.sqrt(len(sample))
upper = sample_mean + z * sample_sd / np.sqrt(len(sample))

print("Sample mean:", sample_mean)
print("95% CI: (", lower, ", ", upper, ")")

```

example 5 : Rice Quality Statistical Analysis & Defect Probability Simulation

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, binom, poisson

# -----
# 1. Load or Simulate Rice Grain Length Data (Real data can be loaded)
# -----
np.random.seed(42)

# Simulating grain lengths (mm) normally distributed
population = np.random.normal(loc=6.5, scale=0.5, size=50000)

print("\n=== RICE QUALITY ANALYSIS PROJECT ===")

# -----
# 2. Draw a Random Sample
# -----

```

```

sample_size = 200
sample = np.random.choice(population, sample_size)

sample_mean = np.mean(sample)
sample_sd = np.std(sample, ddof=1)

print(f"\nSample Size = {sample_size}")
print(f"Sample Mean Grain Length = {sample_mean:.3f} mm")
print(f"Sample Standard Deviation = {sample_sd:.3f} mm")

# -----
# 3. 95% Confidence Interval for Mean
# -----
z = norm.ppf(0.975) # 95% CI
ci_low = sample_mean - z * sample_sd / np.sqrt(sample_size)
ci_high = sample_mean + z * sample_sd / np.sqrt(sample_size)

print(f"\n95% Confidence Interval for Mean Grain Length = ({ci_low:.3f}, {ci_high:.3f}) mm")

# -----
# 4. Probability Distribution Fitting
# -----
mu, sigma = norm.fit(sample)
print(f"\nFitted Normal Distribution: mean={mu:.3f}, sd={sigma:.3f}")

# -----
# 5. Probability a Grain is "Too Short" (< 5.5 mm)
# -----
threshold = 5.5
prob_short = norm.cdf(threshold, mu, sigma)

print(f"\nProbability grain < {threshold} mm = {prob_short:.4f}")

# -----
# 6. Batch Defect Probability
# -----
batch_size = 1000
expected_defects = batch_size * prob_short

```

```

prob_at_most_10_defects = binom.cdf(10, batch_size, prob_short)

print(f"\nExpected defective grains in batch of {batch_size} = {expected_defects:.2f}")
print(f"Probability that batch has  $\leq 10$  defects = {prob_at_most_10_defects:.4f}")

# -----
# 7. Poisson Approximation for Rare Defect Events
# -----
 $\lambda$  = expected_defects
prob_zero_defects = poisson.pmf(0,  $\lambda$ )

print(f"Poisson Approximation: Probability of 0 defects = {prob_zero_defects:.4f}")

# -----
# 8. Monte-Carlo Simulation of Batch Quality
# -----
simulations = 5000
defects = np.random.binomial(batch_size, prob_short, size=simulations)

mean_defects = np.mean(defects)
prob_batch_good = np.mean(defects <= 10)

print(f"\nMonte-Carlo Average Defects = {mean_defects:.2f}")
print(f"Simulated Probability batch  $\leq 10$  defects = {prob_batch_good:.4f}")

# -----
# 9. Plotting Results
# -----

plt.figure(figsize=(15,5))

# 9.1 Histogram with fitted Normal curve
plt.subplot(1,3,1)
plt.hist(sample, bins=20, density=True, alpha=0.6)
x = np.linspace(min(sample), max(sample), 200)
plt.plot(x, norm.pdf(x, mu, sigma), linewidth=2)
plt.title("Sample Grain Lengths\nHistogram + Normal Fit")

```

```
plt.xlabel("Grain Length (mm)")
plt.ylabel("Density")
```

```
# 9.2 Defect Distribution from Monte-Carlo
plt.subplot(1,3,2)
plt.hist(defects, bins=20, alpha=0.7)
plt.axvline(10, color="red", linestyle="--")
plt.title("Monte-Carlo Simulation of Defects")
plt.xlabel("Number of Defects")
plt.ylabel("Frequency")
```

```
# 9.3 CDF Plot of Fitted Distribution
plt.subplot(1,3,3)
plt.plot(x, norm.cdf(x, mu, sigma))
plt.title("CDF of Grain Length")
plt.xlabel("Grain Length (mm)")
plt.ylabel("CDF")
```

```
plt.tight_layout()
plt.show()
```

```
# -----
```

```
# 10. Final Report
```

```
# -----
```

```
print("\n=== FINAL REPORT SUMMARY ===")
print(f"Mean Grain Length (sample): {sample_mean:.3f} mm")
print(f"95% CI: ({ci_low:.3f}, {ci_high:.3f})")
print(f"Probability grain < {threshold} mm: {prob_short:.4f}")
print(f"Expected defects per batch: {expected_defects:.2f}")
print(f"Poisson Probability 0 defects: {prob_zero_defects:.4f}")
print(f"Monte-Carlo Probability  $\leq$  10 defects: {prob_batch_good:.4f}")
print("=====")
```

6. Application :quality control in manufacturing

```
import matplotlib.pyplot as plt
from scipy.stats import bernoulli
import numpy as np
```

```
p = 0.08 # probability defective
```

```
x = [0, 1]
```

```
pmf = bernoulli.pmf(x, p)
```

```
plt.stem(x, pmf, use_line_collection=True)
plt.title("Application 1: Quality Control (p = 0.08)")
plt.xticks([0, 1], ["Not Defective (0)", "Defective (1)"])
plt.ylabel("Probability")
plt.show()
```

7. ★ APPLICATION 2: Website Click-Through Rate (CTR)

A visitor either clicks (1) or does not click (0) an advertisement.

If click-through rate = 5%, then $p=0.05$

```
p = 0.05 # probability user clicks ad
x = [0, 1]
pmf = bernoulli.pmf(x, p)
plt.stem(x, pmf, use_line_collection=True)
plt.title("Application 2: Website CTR (p = 0.05)")
plt.xticks([0, 1], ["No Click (0)", "Click (1)"])
plt.ylabel("Probability")
plt.show()
```

8. A rapid diagnostic test returns either:

- **Positive (1)**
- **Negative (0)**

Assume chance of a positive result = $p = 0.12$.

```
p = 0.12 # probability of positive test
x = [0, 1]
```

```

pmf = bernoulli.pmf(x, p)
plt.stem(x, pmf, use_line_collection=True)
plt.title("Application 3: Medical Test Result (p = 0.12)")
plt.xticks([0, 1], ["Negative (0)", "Positive (1)"])
plt.ylabel("Probability")
plt.show()

```

9.

```

"""

```

Joint PDF demo: $f(x,y) = 2$ for $0 < x < y < 1$ (0 otherwise)

Features:

- numerical check of normalization
- contour + 3D surface plots
- marginal $f_Y(y)$ analytic vs numeric
- rejection sampling from joint pdf
- compute $P(X+Y < 1)$ by sampling and by integration
- visualize conditional $f_{\{X|Y\}}(x|y)$ for a chosen y

```

"""

```

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy import integrate
from scipy.stats import gaussian_kde

# --- 1) Define joint pdf ---
def joint_pdf(x, y):
    """Vectorized joint pdf. x,y can be numpy arrays."""
    # condition:  $0 < x < y < 1$ 
    x = np.asarray(x)
    y = np.asarray(y)
    cond = (x > 0) & (y > 0) & (x < y) & (y < 1)
    out = np.zeros_like(x + y, dtype=float) # broadcasting-safe
    out[cond] = 2.0
    return out

```

```

# scalar wrapper for integrate.dblquad (which expects scalar functions)

```

```

def joint_pdf_scalar(y, x): # dblquad uses order: inner var, outer var
    # note: dblquad passes x then y reversed, so signature (y, x) is convenient
    if 0 < x < y < 1:
        return 2.0
    return 0.0

# --- 2) Numerical check: normalization ---
# integrate over region 0<x<y<1
norm_val, norm_err = integrate.dblquad(joint_pdf_scalar, 0.0, 1.0,
                                       lambda x: 0.0, # lower y bound given x
                                       lambda x: 1.0) # upper y bound (we'll filter by condition inside)
# But because joint_pdf_scalar enforces x<y, dblquad over rectangle is fine.
print(f"Numerical integral over unit square (should be 1): {norm_val:.6f} (err ~ {norm_err:.1e})")

# --- 3) Contour and 3D surface plots ---
N = 300
x = np.linspace(0, 1, N)
y = np.linspace(0, 1, N)
X, Y = np.meshgrid(x, y)
Z = joint_pdf(X, Y)

plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.contourf(X, Y, Z, levels=20, cmap='viridis')
plt.title("Joint PDF Contour: f(x,y)=2 for 0<x<y<1")
plt.xlabel("x")
plt.ylabel("y")
plt.colorbar(label="f(x,y)")

# 3D surface
ax = plt.subplot(1,2,2, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', linewidth=0, antialiased=True)
ax.set_title("Joint PDF Surface")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("f(x,y)")
plt.tight_layout()
plt.show()

```

--- 4) Analytical marginal of Y: $f_Y(y) = \int_0^1 y^2 dx = 2y$ for $0 < y < 1$ ---

def fY_analytic(y):

return 2*y if 0 < y < 1 else 0.0

numeric marginal via integration for a grid of y

ys = np.linspace(0.01, 0.99, 50)

fY_numeric = []

for yy in ys:

val, _ = integrate.quad(lambda xx: joint_pdf(xx, yy), 0.0, 1.0)

fY_numeric.append(val)

Plot analytic vs numeric marginal

plt.figure()

plt.plot(ys, [fY_analytic(v) for v in ys], label='Analytic $f_Y(y)=2y$ ')

plt.plot(ys, fY_numeric, 'o', label='Numeric marginal (integration)', markersize=4)

plt.xlabel('y')

plt.ylabel('f_Y(y)')

plt.title('Marginal of Y: Analytic vs Numeric')

plt.legend()

plt.grid()

plt.show()

--- 5) Rejection sampling from the joint pdf ---

Proposal: uniform on unit square (0,1)x(0,1), with envelope $M = 2$ (max of f)

def sample_from_joint(n_samples=2000):

samples = []

attempts = 0

while len(samples) < n_samples:

attempts += 1

ux = np.random.rand()

uy = np.random.rand()

acceptance prob $f(u)/M$ where $M=2$ and proposal density is 1 over unit square

if joint_pdf(ux, uy) / 2.0 > np.random.rand():

samples.append((ux, uy))

samples = np.array(samples)

print acceptance rate

```

    print(f"Rejection sampling: obtained {n_samples} samples in {attempts} attempts
    (acceptance ~ {n_samples/attempts:.3f})")
    return samples

```

```

samples = sample_from_joint(5000)
sx, sy = samples[:,0], samples[:,1]

```

```

plt.figure(figsize=(6,5))
plt.scatter(sx, sy, s=6, alpha=0.5)
plt.title("Samples from joint distribution (rejection sampling)")
plt.xlabel("x")
plt.ylabel("y")
plt.xlim(0,1); plt.ylim(0,1)
plt.show()

```

```

# --- 6) Verify marginal fY via samples ---
# Empirical density of Y vs analytic
ys_emp = sy
kde = gaussian_kde(ys_emp)
ygrid = np.linspace(0,1,200)
plt.figure()
plt.plot(ygrid, kde(ygrid), label='Empirical KDE of Y from samples')
plt.plot(ygrid, [fY_analytic(v) for v in ygrid], label='Analytic f_Y(y)=2y', lw=2)
plt.xlabel('y'); plt.ylabel('density')
plt.title('Empirical vs Analytic Marginal of Y')
plt.legend(); plt.grid(); plt.show()

```

```

# --- 7) Estimate  $P(X + Y < 1)$  by sampling and by direct integration ---
# Monte Carlo estimate
prob_mc = np.mean((sx + sy) < 1)
print(f"Monte Carlo estimate  $P(X+Y < 1) = \{prob\_mc:.6f\}$ ")

```

```

# Direct integration: integrate over region  $\{0 < x < y < 1, x+y < 1\}$ 
# Integration limits: y from 0 to 1/2 -> x: 0..y (because  $y < 1/2 \Rightarrow y \leq 1-y$ )
# y from 1/2 to 1 -> x: 0..1-y (since  $x < y$  and  $x+y < 1 \Rightarrow x < 1-y < y$ )
def integrand_x(x, y):
    return 2.0 # inside valid region
# integrate piecewise

```

```

l1, _ = integrate.dblquad(lambda xx, yy: integrand_x(xx, yy),
                        0.0, 0.5, # outer y from 0..1/2
                        lambda y: 0.0, # inner x lower bound
                        lambda y: y) # inner x upper bound
l2, _ = integrate.dblquad(lambda xx, yy: integrand_x(xx, yy),
                        0.5, 1.0,
                        lambda y: 0.0,
                        lambda y: 1 - y)
prob_integral = l1 + l2
print(f"Direct integral P(X+Y < 1) = {prob_integral:.6f}")

# --- 8) Conditional density  $f_{\{X|Y\}}(x|y)$  and empirical check ---
# Analytic: for fixed  $y$  ( $0 < y < 1$ ),  $f_{\{X|Y\}}(x|y) = 1/y$  for  $0 < x < y$ 
y0 = 0.7
mask = np.isclose(sy, sy) # trivial placeholder
# select samples with y in small band around y0
band = 0.02
cond_idx = (sy > (y0 - band)) & (sy < (y0 + band))
x_given_y = sx[cond_idx]
print(f"Number of samples with Y in [{y0-band:.2f},{y0+band:.2f}]: {len(x_given_y)}")
if len(x_given_y) > 50:
    plt.figure()
    plt.hist(x_given_y, bins=30, density=True, alpha=0.6, label='Empirical  $f_{\{X|Y\}}(x|y \approx y_0)$ ')
    # analytic conditional (uniform on (0,y0)) plot
    xs_plot = np.linspace(0, y0, 200)
    plt.plot(xs_plot, [1.0/y0]*len(xs_plot), 'r--', lw=2, label=f'Analytic  $f_{\{X|Y\}} = 1/\{y_0\}$ ')
    plt.xlim(0,1)
    plt.xlabel('x')
    plt.ylabel('density')
    plt.title(f'Conditional density  $f_{\{X|Y\}}(x|y \approx \{y_0\})$ ')
    plt.legend()
    plt.show()
else:
    print("Not enough conditional samples in the band to show a smooth histogram. Increase sample size or band.")

# End of script

```

- **10. Part A:** Basic Bayes formula — compute $P(\text{Disease} \mid \text{Positive})$ and see how low prevalence kills positive predictive value (PPP).
- **Part B:** Conjugate updating (Beta prior $\rightarrow$ Beta posterior) for Bernoulli trials — clean, analytic, easy to compute and plot.
- **Part C:** Monte Carlo shows how different priors influence posterior uncertainty when sample size is limited.
- **Part D:** Naive Bayes classification frames Bayes theorem for multiple features (assumes conditional independence). You'll inspect AUC and calibration to see how the predicted probabilities behave as posteriors.
- **Part E:** Small experiments to show how one observation moves a prior to a posterior (intuition).

```
# Bayes_Project.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import beta, bernoulli
from sklearn.datasets import load_breast_cancer
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.calibration import calibration_curve
from sklearn.metrics import roc_auc_score, roc_curve, accuracy_score

sns.set(style="whitegrid", palette="muted")

# -----
# Part A: Analytic Bayes - single test
# -----
def analytic_bayes(prevalence, sensitivity, specificity):
    """
    Returns  $P(\text{Disease} \mid \text{Positive Test})$  using Bayes' theorem.
    prevalence =  $P(D)$ 
    sensitivity =  $P(+ \mid D)$ 
    specificity =  $P(- \mid \text{not } D)$ 
    """
```

```

p = prevalence
sens = sensitivity
spec = specificity
P_pos = p * sens + (1 - p) * (1 - spec)
posterior = p * sens / P_pos
return posterior

# Example numbers
prevalence = 0.01  # 1% disease prevalence
sensitivity = 0.95  # P(+ | D)
specificity = 0.92  # P(- | not D)

post = analytic_bayes(prevalence, sensitivity, specificity)
print("Analytic Bayes: P(D | +) = {:.4f} {:.2%}".format(post, post))

# -----
# Part B: Beta-Binomial sequential updating
# -----
def bayesian_update_beta(prior_a, prior_b, positives, negatives):
    """
    Beta(a,b) prior updated with observed positives/negatives (Bernoulli trials)
    Returns posterior Beta(a + positives, b + negatives)
    """
    a_post = prior_a + positives
    b_post = prior_b + negatives
    return a_post, b_post

# Choose a prior (weakly informative and informative examples)
priors = {
    "uninformative (1,1)": (1,1),
    "weakly informative (2,20)": (2,20),  # favors small p
    "informative (50,950)": (50,950)  # strong prior ~ 5% mean
}

# Simulate a batch of tests
np.random.seed(0)
true_p = 0.03  # true probability of disease (3%)
N = 500

```

```

data = bernoulli.rvs(true_p, size=N)
positives = data.sum()
negatives = N - positives
print(f"Simulated batch: N={N}, positives={positives}, negatives={negatives}")

# Update priors and plot posterior distributions
y = np.linspace(0, 1, 1000)
plt.figure(figsize=(10,6))
for name, (a0,b0) in priors.items():
    a_post, b_post = bayesian_update_beta(a0, b0, positives, negatives)
    post_pdf = beta.pdf(y, a_post, b_post)
    prior_pdf = beta.pdf(y, a0, b0)
    plt.plot(y, post_pdf, label=f"{name} posterior (a={a_post},b={b_post})")
    plt.plot(y, prior_pdf, linestyle="--", alpha=0.6, label=f"{name} prior")

plt.title("Prior vs Posterior (Beta-Binomial) after observing batch")
plt.xlabel("probability of disease (p)")
plt.ylabel("density")
plt.legend(ncol=1, bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()
plt.show()

# Print posterior summaries
for name, (a0,b0) in priors.items():
    a_post, b_post = bayesian_update_beta(a0, b0, positives, negatives)
    mean_post = a_post / (a_post + b_post)
    ci_low, ci_high = beta.ppf([0.025, 0.975], a_post, b_post)
    print(f"{name}: posterior mean={mean_post:.4f}, 95% CI=({ci_low:.4f}, {ci_high:.4f})")

# -----
# Part C: Monte Carlo sensitivity to priors
# -----
def monte_carlo_prior_sensitivity(prior_list, true_p=0.03, N=500, repeats=1000):
    """
    For each prior, simulate 'repeats' experiments of size N from true_p,
    compute posterior mean for each repeat, then summarize.
    """
    results = {}

```

```

for (a0,b0) in prior_list:
    means = []
    for _ in range(repeats):
        data = bernoulli.rvs(true_p, size=N)
        pos = data.sum()
        a_post, b_post = a0 + pos, b0 + (N - pos)
        means.append(a_post / (a_post + b_post))
    results[(a0,b0)] = np.array(means)
return results

prior_list = [(1,1), (2,20), (50,950)]
results = monte_carlo_prior_sensitivity(prior_list, true_p=true_p, N=N, repeats=500)

plt.figure(figsize=(8,5))
for prior, arr in results.items():
    sns.kdeplot(arr, label=f"prior={prior}")
plt.axvline(true_p, color='k', linestyle='--', label='true p')
plt.title("Monte Carlo: Posterior mean distribution under different priors")
plt.xlabel("posterior mean")
plt.legend()
plt.show()

# -----
# Part D: Naive Bayes classification on real dataset (breast cancer)
# -----
data = load_breast_cancer()
X = data.data
y = data.target # 0=malignant, 1=benign in sklearn (check doc; we will treat 1 as positive class
for demonstration)
# Remap so disease=1 for clarity (we want positive=malignant typically). Let's set disease=1 if
malignant:
# sklearn: target_names = ['malignant' 'benign'] and target = 0 (malignant), 1 (benign). So flip:
y = 1 - y # now y=1 => malignant (disease), y=0 => benign (no disease)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

clf = GaussianNB()

```

```

clf.fit(X_train, y_train)
y_proba = clf.predict_proba(X_test)[:,-1] # P(disease | features) as given by model
y_pred = clf.predict(X_test)

auc = roc_auc_score(y_test, y_proba)
acc = accuracy_score(y_test, y_pred)
print(f"Gaussian Naive Bayes: AUC={auc:.3f}, Accuracy={acc:.3f}")

# ROC curve
fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC={auc:.3f}")
plt.plot([0,1],[0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC - Gaussian Naive Bayes on Breast Cancer")
plt.legend()
plt.show()

# Calibration plot (reliability diagram)
prob_true, prob_pred = calibration_curve(y_test, y_proba, n_bins=10)
plt.figure(figsize=(6,5))
plt.plot(prob_pred, prob_true, marker='o', label='Naive Bayes')
plt.plot([0,1],[0,1], 'k--', label='Perfectly calibrated')
plt.xlabel("Predicted probability")
plt.ylabel("Observed frequency")
plt.title("Calibration plot")
plt.legend()
plt.show()

# Show example: how Bayes theorem interprets a positive classifier probability
# pick a test sample and display feature-probability and Bayes analytic idea:
idx = 0
print("Example test sample predicted P(disease) = {:.3f}, true label = {}".format(y_proba[idx],
y_test[idx]))
# Note: this model's predicted P is an estimate; Bayes theorem is underlying concept.

# -----

```

```

# Part E: Small interactive-ish experiments (non-interactive here)
# -----
# 1) Prior sensitivity: show how posterior for a single positive/negative changes
def show_posterior_after_single_obs(prior_a, prior_b, obs):
    """
    obs: 1 for positive (disease observed), 0 for negative.
    """
    a_post = prior_a + obs
    b_post = prior_b + (1 - obs)
    y = np.linspace(0,1,500)
    plt.plot(y, beta.pdf(y, prior_a, prior_b), linestyle='--', label='prior')
    plt.plot(y, beta.pdf(y, a_post, b_post), label='posterior after obs={}'.format(obs))
    plt.title(f"Prior (a={prior_a},b={prior_b}) -> posterior after one obs")
    plt.xlabel("p")
    plt.ylabel("density")
    plt.legend()
    plt.show()

show_posterior_after_single_obs(1,1,1) # uninformative prior, one positive
show_posterior_after_single_obs(50,950,1) # informative prior, one positive

# -----
# End of project script.
# -----

```